

```

// File: app/src/main/java/com/liudong/bookread/MainActivity.kt
package com.liudong.bookread
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.ui.Modifier
import com.liudong.bookread.ui.navigation.AppNavigation
import com.liudong.bookread.ui.theme.BookReadTheme
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            BookReadTheme {
                Surface(
                    modifier = Modifier.fillMaxSize(),
                    color = MaterialTheme.colorScheme.background
                ) {
                    AppNavigation()
                }
            }
        }
    }
}

// File: app/src/main/java/com/liudong/bookread/ui/navigation/AppNavigation.kt
package com.liudong.bookread.ui.navigation
import androidx.compose.runtime.Composable
import androidx.lifecycle.viewmodel.compose.viewModel
import androidx.navigation.NavType
import androidx.navigation.compose.NavHost
import androidx.navigation.compose.composable
import androidx.navigation.compose.rememberNavController
import androidx.navigation.navArgument
import com.liudong.bookread.ui.detail.TextbookDetailScreen
import com.liudong.bookread.ui.home.HomeScreen
import com.liudong.bookread.ui.reader.ReaderScreen
import com.liudong.bookread.viewmodel.TextbookViewModel
@Composable
fun AppNavigation() {
    val navController = rememberNavController()
    val viewModel: TextbookViewModel = viewModel()
    NavHost(navController = navController, startDestination = "home") {
        composable("home") {
            HomeScreen(viewModel = viewModel, navController = navController)
        }
    }
}

```

```

composable(
    "detail/{textbookId}",
    arguments = listOf(navArgument("textbookId") { type = NavType.StringType })
) { backStackEntry ->
    val textbookId = backStackEntry.arguments?.getString("textbookId")!!
    TextbookDetailScreen(viewModel, textbookId, navController)
}

composable(
    "reader/{textbookId}/{unitId}/{pageId}",
    arguments = listOf(
        navArgument("textbookId") { type = NavType.StringType },
        navArgument("unitId") { type = NavType.StringType },
        navArgument("pageId") { type = NavType.StringType }
    )
) { backStackEntry ->
    val textbookId = backStackEntry.arguments?.getString("textbookId")!!
    val unitId = backStackEntry.arguments?.getString("unitId")!!
    val pageId = backStackEntry.arguments?.getString("pageId")!!
    ReaderScreen(viewModel, textbookId, unitId, pageId, navController)
}
}
}

// File: app/src/main/java/com/liudong/bookread/model/Models.kt
package com.liudong.bookread.model

import kotlinx.serialization.Serializable
import java.util.UUID

@Serializable
data class WordAnnotation(
    val id: String = UUID.randomUUID().toString(),
    var word: String,
    var phonetic: String,
    var normalizedX: Double,
    var normalizedY: Double,
    var normalizedWidth: Double,
    var normalizedHeight: Double
)

@Serializable
data class TextbookPage(
    val id: String = UUID.randomUUID().toString(),
    var name: String,
    var imagePath: String,
    var order: Int,
    var annotations: List<WordAnnotation> = emptyList()
)

@Serializable
data class TextbookUnit(
    val id: String = UUID.randomUUID().toString(),
    var name: String,
    var order: Int,
    var pages: List<TextbookPage> = emptyList()
)

```

```

)

@Serializable
data class Textbook(
    val id: String = UUID.randomUUID().toString(),
    var name: String,
    var coverImagePath: String? = null,
    val createdAt: Long = System.currentTimeMillis(),
    var units: List<TextbookUnit> = emptyList()
)

// File: app/src/main/java/com/liudong/bookread/viewmodel/TextbookViewModel.kt
package com.liudong.bookread.viewmodel

import android.app.Application
import android.graphics.Bitmap
import android.graphics.BitmapFactory
import androidx.lifecycle.AndroidViewModel
import androidx.lifecycle.viewModelScope
import com.liudong.bookread.R
import com.liudong.bookread.model.Textbook
import com.liudong.bookread.model.TextbookPage
import com.liudong.bookread.model.TextbookUnit
import com.liudong.bookread.model.WordAnnotation
import com.liudong.bookread.service.DataStoreService
import com.liudong.bookread.service.OCRService
import com.liudong.bookread.service.PhoneticDictionaryService
import com.liudong.bookread.service.SpeechService
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.flow.first
import kotlinx.coroutines.launch
import java.util.UUID

class TextbookViewModel(application: Application) : AndroidViewModel(application) {
    private val app = getApplication<Application>()
    private val context get() = app.applicationContext
    private val _textbooks = MutableStateFlow<List<Textbook>>(emptyList())
    val textbooks: StateFlow<List<Textbook>> = _textbooks.asStateFlow()
    private val _isProcessing = MutableStateFlow(false)
    val isProcessing: StateFlow<Boolean> = _isProcessing.asStateFlow()
    private val _processingProgress = MutableStateFlow("")
    val processingProgress: StateFlow<String> = _processingProgress.asStateFlow()

    init {
        viewModelScope.launch {
            loadTextbooks()
            setupSamplePages()
        }

        PhoneticDictionaryService.loadDictionary(context)
        SpeechService.initialize(context)
    }

    private suspend fun loadTextbooks() {
        _textbooks.value = DataStoreService.loadTextbooks(context)
    }

```

```

}

private fun saveTextbooks() {
    viewModelScope.launch {
        DataStoreService.saveTextbooks(context, _textbooks.value)
    }
}

private fun setupSamplePages() {
    // 若示例课本已存在：检查示例页是否已完成 ocr（上次启动可能中断），未完成则补跑
    val existingSample = _textbooks.value.find { it.name == "示例课本" }
    if (existingSample != null) {
        val samplePage = existingSample.units
            .firstOrNull { it.name == "Unit 1 Hello" }
            ?.pages?.firstOrNull()
        if (samplePage != null && samplePage.annotations.isEmpty()) {
            val demoBitmap = DataStoreService.loadImage(context, samplePage.imagePath)
            if (demoBitmap != null) {
                processDemoImage(demoBitmap, samplePage.id)
            }
        }
        return
    }
    // 从资源加载 demo 图片
    val demoBitmap = BitmapFactory.decodeResource(app.resources, R.drawable.demo)
    ?: run {
        android.util.Log.w("TextbookViewModel", "未找到 demo 图片资源")
        return
    }
    // 保存到应用私有目录
    val pageId = UUID.randomUUID().toString()
    val imageFilename = DataStoreService.saveImage(context, demoBitmap, pageId)
    ?: return
    // 创建示例页面（先创建空标注，后台 OCR 完成后自动更新）
    val samplePage = TextbookPage(
        id = pageId,
        name = "示例页",
        imagePath = imageFilename,
        order = 0,
        annotations = emptyList()
    )
    val sampleUnit = TextbookUnit(name = "Unit 1 Hello", order = 0, pages = listOf(samplePage))
    val sampleTextbook = Textbook(name = "示例课本", units = listOf(sampleUnit))
    _textbooks.value = listOf(sampleTextbook) + _textbooks.value
    saveTextbooks()
    // 后台异步 OCR 处理 demo 图片
    processDemoImage(demoBitmap, pageId)
}

/// 后台对 demo 图片执行 OCR，完成后自动更新示例页标注
private fun processDemoImage(bitmap: Bitmap, pageId: String) {
    viewModelScope.launch {
        val ocrResults = try {

```

```

OCRService.recognizeText(bitmap).first()
} catch (e: Exception) {
    android.util.Log.e("TextbookViewModel", "Demo 图片 OCR 识别失败", e)
    return@launch
}

val annotations = ocrResults.map { result ->
    WordAnnotation(
        word = result.text,
        phonetic = PhoneticDictionaryService.lookup(result.text) ?: "",
        normalizedX = result.boundingBox.left.toDouble() / bitmap.width,
        normalizedY = 1.0 - (result.boundingBox.bottom.toDouble() / bitmap.height),
        normalizedWidth = result.boundingBox.width().toDouble() / bitmap.width,
        normalizedHeight = result.boundingBox.height().toDouble() / bitmap.height
    )
}

// 更新示例课本的页面标注
_textbooks.value = _textbooks.value.map { tb ->
    if (tb.name == "示例课本") {
        tb.copy(units = tb.units.map { unit ->
            if (unit.name == "Unit 1 Hello") {
                unit.copy(pages = unit.pages.map { p ->
                    if (p.id == pageId) p.copy(annotations = annotations) else p
                })
            } else unit
        })
    } else tb
}

saveTextbooks()
}

fun addTextbook(name: String): Textbook {
    val textbook = Textbook(name = name)
    _textbooks.value = _textbooks.value + textbook
    saveTextbooks()
    return textbook
}

fun deleteTextbook(textbook: Textbook) {
    textbook.units.flatMap { it.pages }.forEach { page ->
        DataStoreService.deleteImage(context, page.imagePath)
    }
    _textbooks.value = _textbooks.value.filter { it.id != textbook.id }
    saveTextbooks()
}

fun addPage(
    textbookId: String,
    unitId: String,
    bitmap: Bitmap,
    pageName: String,
    onComplete: (TextbookPage?) -> Unit
) {

```

```

viewModelScope.launch {
    _isProcessing.value = true
    _processingProgress.value = "正在识别文字..."
    try {
        // 校验课本与单元存在，避免产生孤儿图片与无效导航
        val targetExists = _textbooks.value.any { tb ->
            tb.id == textbookId && tb.units.any { it.id == unitId }
        }
        if (!targetExists) {
            onComplete(null)
            return@launch
        }
        val pageId = UUID.randomUUID().toString()
        val imageFilename = DataStoreService.saveImage(context, bitmap, pageId)
        ?: run {
            onComplete(null)
            return@launch
        }
        val ocrResults = OCRService.recognizeText(bitmap).first()
        _processingProgress.value = "正在生成音标..."
        val annotations = ocrResults.map { result ->
            WordAnnotation(
                word = result.text,
                phonetic = PhoneticDictionaryService.lookup(result.text) ?: "",
                normalizedX = result.boundingBox.left.toDouble() / bitmap.width,
                normalizedY = 1.0 - (result.boundingBox.bottom.toDouble() / bitmap.height),
                normalizedWidth = result.boundingBox.width().toDouble() / bitmap.width,
                normalizedHeight = result.boundingBox.height().toDouble() / bitmap.height
            )
        }
        val page = TextbookPage(
            id = pageId,
            name = pageName,
            imagePath = imageFilename,
            order = 0,
            annotations = annotations
        )
        val updated = _textbooks.value.map { tb ->
            if (tb.id == textbookId) {
                tb.copy(units = tb.units.map { unit ->
                    if (unit.id == unitId) unit.copy(pages = unit.pages + page) else unit
                })
            } else tb
        }
        _textbooks.value = updated
        saveTextbooks()
        onComplete(page)
    } catch (e: Exception) {
        android.util.Log.e("TextbookViewModel", "添加页面识别失败", e)
        onComplete(null)
    }
}

```

```

} finally {
    _isProcessing.value = false
    _processingProgress.value = ""
}
}
}

fun updateAnnotations(textbookId: String, unitId: String, pageId: String, annotations: List<WordAnnotation>) {
    val updated = _textbooks.value.map { tb ->
        if (tb.id == textbookId) {
            tb.copy(units = tb.units.map { unit ->
                if (unit.id == unitId) {
                    unit.copy(pages = unit.pages.map { p ->
                        if (p.id == pageId) p.copy(annotations = annotations) else p
                    })
                } else unit
            })
        } else tb
    }
    _textbooks.value = updated
    saveTextbooks()
}

fun deletePage(textbookId: String, unitId: String, page: TextbookPage) {
    DataStoreService.deleteImage(context, page.imagePath)
    val updated = _textbooks.value.map { tb ->
        if (tb.id == textbookId) {
            tb.copy(units = tb.units.map { unit ->
                if (unit.id == unitId) unit.copy(pages = unit.pages.filter { it.id != page.id })
            })
        } else tb
    }
    _textbooks.value = updated
    saveTextbooks()
}

fun renamePage(textbookId: String, unitId: String, page: TextbookPage, newName: String) {
    val updated = _textbooks.value.map { tb ->
        if (tb.id == textbookId) {
            tb.copy(units = tb.units.map { unit ->
                if (unit.id == unitId) {
                    unit.copy(pages = unit.pages.map { p ->
                        if (p.id == page.id) p.copy(name = newName) else p
                    })
                } else unit
            })
        } else tb
    }
    _textbooks.value = updated
    saveTextbooks()
}

fun addUnit(textbookId: String, name: String): TextbookUnit? {

```

```

val textbook = _textbooks.value.find { it.id == textbookId } ?: return null
val order = textbook.units.size
val unit = TextbookUnit(name = name, order = order)
val updated = _textbooks.value.map { tb ->
    if (tb.id == textbookId) tb.copy(units = tb.units + unit) else tb
}
_textbooks.value = updated
saveTextbooks()
return unit
}

fun reprocessPage(textbookId: String, unitId: String, page: TextbookPage, bitmap: Bitmap, onComplete: (List<WordAnnotation>)
-> Unit) {
    viewModelScope.launch {
        _isProcessing.value = true
        _processingProgress.value = "重新识别中..."
        try {
            val ocrResults = OCRService.recognizeText(bitmap).first()
            val annotations = ocrResults.map { result ->
                WordAnnotation(
                    word = result.text,
                    phonetic = PhoneticDictionaryService.lookup(result.text) ?: "",
                    normalizedX = result.boundingBox.left.toDouble() / bitmap.width,
                    normalizedY = 1.0 - (result.boundingBox.bottom.toDouble() / bitmap.height),
                    normalizedWidth = result.boundingBox.width().toDouble() / bitmap.width,
                    normalizedHeight = result.boundingBox.height().toDouble() / bitmap.height
                )
            }
            val updated = _textbooks.value.map { tb ->
                if (tb.id == textbookId) {
                    tb.copy(units = tb.units.map { u ->
                        if (u.id == unitId) {
                            u.copy(pages = u.pages.map { p ->
                                if (p.id == page.id) p.copy(annotations = annotations) else p
                            })
                        } else u
                    })
                } else tb
            }
            _textbooks.value = updated
            saveTextbooks()
            onComplete(annotations)
        } catch (e: Exception) {
            android.util.Log.e("TextbookViewModel", "重新识别失败", e)
            // 失败时保留页面原有标注，避免 ui 被清空后再次保存造成数据丢失
            onComplete(page.annotations)
        } finally {
            _isProcessing.value = false
            _processingProgress.value = ""
        }
    }
}

```

```

}

// File: app/src/main/java/com/liudong/bookread/service/DataStoreService.kt
package com.liudong.bookread.service

import android.content.Context
import android.graphics.Bitmap
import android.graphics.BitmapFactory
import android.util.Log
import androidx.datastore.preferences.core.edit
import androidx.datastore.preferences.core.stringPreferencesKey
import androidx.datastore.preferences.preferencesDataStore
import com.liudong.bookread.model.Textbook
import kotlinx.coroutines.flow.first
import kotlinx.serialization.encodeToString
import kotlinx.serialization.json.Json
import java.io.File

private val Context.dataStore by preferencesDataStore(name = "app_preferences")

object DataStoreService {

    private const val TEXTBOOKS_KEY = "saved_textbooks"

    private val json = Json { ignoreUnknownKeys = true }

    suspend fun saveTextbooks(context: Context, textbooks: List<Textbook>) {
        val jsonStr = json.encodeToString(textbooks)
        context.dataStore.edit { prefs ->
            prefs[stringPreferencesKey(TEXTBOOKS_KEY)] = jsonStr
        }
    }

    suspend fun loadTextbooks(context: Context): List<Textbook> {
        val prefs = context.dataStore.data.first()
        val jsonStr = prefs[stringPreferencesKey(TEXTBOOKS_KEY)] ?: return emptyList()
        return try {
            json.decodeFromString(jsonStr)
        } catch (e: Exception) {
            Log.e("DataStore", "加载课本数据失败", e)
            emptyList()
        }
    }

    fun saveImage(context: Context, bitmap: Bitmap, pageId: String): String? {
        val filename = "$pageId.jpg"
        val file = File(context.filesDir, filename)
        return try {
            file.outputStream().use { out ->
                bitmap.compress(Bitmap.CompressFormat.JPEG, 80, out)
            }
            filename
        } catch (e: Exception) {
            Log.e("DataStore", "保存图片失败", e)
            null
        }
    }

    fun loadImage(context: Context, filename: String): Bitmap? {
        val file = File(context.filesDir, filename)
    }

```

```

return if (file.exists()) BitmapFactory.decodeFile(file.absolutePath) else null
}

fun deleteImage(context: Context, filename: String) {
File(context.filesDir, filename).delete()
}
}

// File: app/src/main/java/com/liudong/bookread/service/OCRService.kt
package com.liudong.bookread.service

import android.graphics.Bitmap
import android.graphics.Rect
import com.google.mlkit.vision.common.InputImage
import com.google.mlkit.vision.text.TextRecognition
import com.google.mlkit.vision.text.latin.TextRecognizerOptions
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.flow
import kotlinx.coroutines.flow.flowOn
import kotlinx.coroutines.suspendCancellableCoroutine
import kotlin.coroutines.resume
import kotlin.coroutines.resumeWithException

object OCRService {
data class OCRResult(
val text: String,
val boundingBox: Rect
)

private val recognizer = TextRecognition.getClient(TextRecognizerOptions.DEFAULT_OPTIONS)

fun recognizeText(bitmap: Bitmap): Flow<List<OCRResult>> = flow {
val image = InputImage.fromBitmap(bitmap, 0)
val result = recognizer.process(image).await()
val results = mutableListOf<OCRResult>()
for (block in result.textBlocks) {
for (line in block.lines) {
val text = line.text.trim()
if (!shouldKeep(text)) continue
val words = text.split(Regex("\\s+")).filter { it.isNotEmpty() }
if (words.size > 1) {
for (element in line.elements) {
val word = element.text.trim()
if (shouldKeep(word)) {
results.add(OCRResult(word, element.boundingBox ?: line.boundingBox!!))
}
}
} else {
results.add(OCRResult(text, line.boundingBox!!))
}
}
}

emit(results)
}.flowOn(Dispatchers.IO)

private fun shouldKeep(text: String): Boolean {

```

```

if (text.all { it.isDigit() }) return false
if (text.length < 2 && text != "I" && text != "a") return false
val letters = text.filter { it.isLetter() }
return letters.isNotEmpty()
}

private suspend fun <T> com.google.android.gms.tasks.Task<T>.await(): T =
suspendCancellableCoroutine { cont ->
addOnSuccessListener { cont.resume(it) }
addOnFailureListener { cont.resumeWithException(it) }
}
}

// File: app/src/main/java/com/liudong/bookread/service/PhoneticDictionaryService.kt
package com.liudong.bookread.service

import android.content.Context
import android.util.Log
import kotlinx.serialization.encodeToString
import kotlinx.serialization.json.Json

object PhoneticDictionaryService {

private var phoneticDict: MutableMap<String, String> = mutableMapOf()
private var meaningDict: MutableMap<String, String> = mutableMapOf()
private const val USER_EXTENSIONS_KEY = "user_phonetic_extensions"
private val json = Json { ignoreUnknownKeys = true }

fun loadDictionary(context: Context) {
try {
context.assets.open("phonetic_dictionary.json").use { stream ->
val content = stream.bufferedReader().readText()
val map: Map<String, Map<String, String>> = json.decodeFromString(content)
for ((word, info) in map) {
val key = word.lowercase()
phoneticDict[key] = info["phonetic"] ?: ""
meaningDict[key] = info["meaning"] ?: ""
}
}
} catch (e: Exception) {
Log.w("PhoneticDict", "基础词库加载失败或不存在", e)
}

val prefs = context.getSharedPreferences("phonetic_prefs", Context.MODE_PRIVATE)
val saved = prefs.getString(USER_EXTENSIONS_KEY, null)
saved?.let {
try {
val extensions: Map<String, String> = json.decodeFromString(it)
phoneticDict.putAll(extensions.mapKeys { entry -> entry.key.lowercase() })
} catch (e: Exception) {
Log.w("PhoneticDict", "用户扩展词库加载失败", e)
}
}

Log.d("PhoneticDict", "音标词库加载完成, 共 ${phoneticDict.size} 个单词")
}

fun lookup(word: String): String? {
val clean = word.lowercase().trim { it in ".,:;!?'\"()[]{}" }

```

```

return phoneticDict[clean]?.takeIf { it.isNotEmpty() }
}

fun lookupMeaning(word: String): String? {
val clean = word.lowercase().trim { it in ".,,:!?'\"()[]{}" }
return meaningDict[clean]?.takeIf { it.isNotEmpty() }
}

fun addCustomPhonetic(context: Context, word: String, phonetic: String) {
val key = word.lowercase()
phoneticDict[key] = phonetic
saveUserExtensions(context)
}

private fun saveUserExtensions(context: Context) {
val prefs = context.getSharedPreferences("phonetic_prefs", Context.MODE_PRIVATE)
val mapToSave: Map<String, String> = phoneticDict
prefs.edit().putString(USER_EXTENSIONS_KEY, json.encodeToString(mapToSave)).apply()
}
}

// File: app/src/main/java/com/liudong/bookread/service/SpeechService.kt
package com.liudong.bookread.service

import android.content.Context
import android.os.Build
import android.os.Handler
import android.os.Looper
import android.speech.tts.TextToSpeech
import android.speech.tts.UtteranceProgressListener
import android.util.Log
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import java.util.Locale

object SpeechService {
private var tts: TextToSpeech? = null
private var isInitialized = false
private val _isSpeaking = MutableStateFlow(false)
val isSpeaking: StateFlow<Boolean> = _isSpeaking.asStateFlow()
private val _currentWord = MutableStateFlow<String?>(null)
val currentWord: StateFlow<String?> = _currentWord.asStateFlow()
private val _isAvailable = MutableStateFlow(false)
val isAvailable: StateFlow<Boolean> = _isAvailable.asStateFlow()
private val _errorMessage = MutableStateFlow<String?>(null)
val errorMessage: StateFlow<String?> = _errorMessage.asStateFlow()
var accent: String = "en-US"
var speechRate: Float = 1.0f
// 记录已经尝试过的引擎，避免死循环
private val attemptedEngines = mutableSetOf<String>()
fun initialize(context: Context) {
if (isInitialized) return
attemptedEngines.clear()
val mainHandler = Handler(Looper.getMainLooper())
val tempTts = TextToSpeech(context) { status ->

```

```

mainHandler.post {
    val currentTts = tts ?: return@post
    handleInitResult(currentTts, status, context)
}
}

tts = tempTts
}

private fun handleInitResult(currentTts: TextToSpeech, status: Int, context: Context) {
    val engines = currentTts.engines
    Log.i("SpeechService", "设备上共有 ${engines.size} 个 TTS 引擎:")
    engines.forEach { engine ->
        Log.i("SpeechService", " - ${engine.name} (label=${engine.label})")
    }

    if (status == TextToSpeech.SUCCESS) {
        trySetupLanguage(currentTts, context)
        return
    }

    // 默认引擎初始化失败, 尝试其他已安装引擎
    Log.e("SpeechService", "默认 TTS 引擎初始化失败, status=$status, 尝试切换引擎...")
    val nextEngine = engines.firstOrNull { it.name !in attemptedEngines }
    if (nextEngine != null) {
        attemptedEngines.add(nextEngine.name)
        Log.i("SpeechService", "尝试切换到引擎: ${nextEngine.name}")
        currentTts.shutdown()
        val mainHandler = Handler(Looper.getMainLooper())
        val newTts = TextToSpeech(context, { newStatus ->
            mainHandler.post {
                val latestTts = tts ?: return@post
                handleInitResult(latestTts, newStatus, context)
            }
        }, nextEngine.name)
        tts = newTts
        return
    }

    // 所有引擎都试过了, 还是失败
    Log.e("SpeechService", "所有 TTS 引擎均初始化失败")
    _errorMessage.value = "语音引擎无法启动, 请前往「设置 → 辅助功能 → 文字转语音」将科大讯飞设为默认引擎"
    _isAvailable.value = false
    currentTts.shutdown()
}

private fun trySetupLanguage(currentTts: TextToSpeech, context: Context) {
    val locale = Locale.US
    val langResult = currentTts.setLanguage(locale)
    Log.i("SpeechService", "setLanguage(Locale.US) 返回: $langResult")
    if (langResult == TextToSpeech.LANG_MISSING_DATA) {
        Log.e("SpeechService", "英语语音数据缺失")
        _errorMessage.value = "英语语音数据缺失, 请在系统设置中下载语音包"
        _isAvailable.value = false
        currentTts.shutdown()
    }
    return
}

```

```

}

if (langResult == TextToSpeech.LANG_NOT_SUPPORTED) {
    Log.e("SpeechService", "当前 TTS 引擎不支持英语")
    _errorMessage.value = "当前语音引擎不支持英语"
    _isAvailable.value = false
    currentTts.shutdown()
    return
}

initialized = true
_isAvailable.value = true
_errorMessage.value = null
currentTts.setOnUtteranceProgressListener(object : UtteranceProgressListener() {
    override fun onStart(utteranceId: String?) {
        _isSpeaking.value = true
    }
    override fun onDone(utteranceId: String?) {
        _isSpeaking.value = false
        _currentWord.value = null
    }
    override fun onError(utteranceId: String?) {
        _isSpeaking.value = false
        _currentWord.value = null
    }
})
Log.i("SpeechService", "TTS 初始化成功, 使用引擎: ${currentTts.defaultEngine}")

fun speak(word: String) {
    if (!initialized || !_isAvailable.value) {
        Log.w("SpeechService", "TTS 不可用, 跳过发音: $word")
        return
    }
    tts?.stop()
    _currentWord.value = word
    _isSpeaking.value = true
    val locale = when (accent) {
        "en-GB" -> Locale.UK
        else -> Locale.US
    }
    tts?.language = locale
    tts?.setSpeechRate(speechRate)
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.LOLLIPOP) {
        tts?.speak(word, TextToSpeech.QUEUE_FLUSH, null, word)
    }
}

fun stop() {
    tts?.stop()
    _isSpeaking.value = false
    _currentWord.value = null
}

fun shutdown() {

```

```

tts?.shutdown()
tts = null
isInitialized = false
_isAvailable.value = false
}
}

// File: app/src/main/java/com/liudong/bookread/ui/home/HomeScreen.kt
package com.liudong.bookread.ui.home

import android.graphics.Bitmap
import android.net.Uri
import androidx.activity.compose.rememberLauncherForActivityResult
import androidx.activity.result.PickVisualMediaRequest
import androidx.activity.result.contract.ActivityResultContracts
import androidx.compose.foundation.Image
import androidx.compose.foundation.background
import androidx.compose.foundation.border
import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.PaddingValues
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.AddCircle
import androidx.compose.material.icons.filled.ArrowForward
import androidx.compose.material.icons.filled.CameraAlt
import androidx.compose.material.icons.filled.CheckCircle
import androidx.compose.material.icons.filled.KeyboardArrowRight
import androidx.compose.material.icons.filled.MenuBook
import androidx.compose.material.icons.filled.PhotoCamera
import androidx.compose.material.icons.filled.PhotoLibrary
import androidx.compose.material3.AlertDialog
import androidx.compose.material3.Button
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.ModalBottomSheet

```

```

import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Text
import androidx.compose.material3.TextButton
import androidx.compose.material3.TopAppBar
import androidx.compose.material3.rememberModalBottomSheetState
import androidx.compose.runtime.Composable
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.draw.shadow
import androidx.compose.ui.graphics.Brush
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.asImageBitmap
import androidx.compose.ui.layout.ContentScale
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.core.content.FileProvider
import androidx.navigation.NavController
import com.liudong.bookread.model.Textbook
import com.liudong.bookread.model.TextbookPage
import com.liudong.bookread.model.TextbookUnit
import com.liudong.bookread.service.DataStoreService
import com.liudong.bookread.ui.components.getBitmapFromUri
import com.liudong.bookread.viewmodel.TextbookViewModel
import java.io.File
import java.text.SimpleDateFormat
import java.util.Date
import java.util.Locale
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun HomeScreen(
    viewModel: TextbookViewModel,
    navController: NavController
) {
    val textbooks by viewModel.textbooks.collectAsState()
    val isProcessing by viewModel.isProcessing.collectAsState()
    val processingProgress by viewModel.processingProgress.collectAsState()
    var showAddDialog by remember { mutableStateOf(false) }
    var newTextbookName by remember { mutableStateOf("") }
    // 快速开始相关状态
    var showQuickStartSheet by remember { mutableStateOf(false) }
    var quickStartTextbookId by remember { mutableStateOf<String?>(null) }

```

```

var quickStartUnitId by remember { mutableStateOf<String?>(null) }
var tempImageUri by remember { mutableStateOf<Uri?>(null) }
val context = LocalContext.current
// 示例课本的 demo 页面数据
val demoTextbook = textbooks.find { it.name == "示例课本" }
val demoUnit = demoTextbook?.units?.firstOrNull()
val demoPage = demoUnit?.pages?.firstOrNull()
val cameraLauncher = rememberLauncherForActivityResult(
    ActivityResultContracts.TakePicture()
) { success ->
    if (success && tempImageUri != null && quickStartTextbookId != null && quickStartUnitId != null) {
        val bitmap = getBitmapFromUri(context, tempImageUri!!)
        bitmap?.let {
            processQuickStartImage(
                viewModel = viewModel,
                bitmap = it,
                textbookId = quickStartTextbookId!!,
                unitId = quickStartUnitId!!,
                navController = navController
            )
        }
        tempImageUri = null
    }
}

val photoPicker = rememberLauncherForActivityResult(
    ActivityResultContracts.PickVisualMedia()
) { uri ->
    if (uri != null && quickStartTextbookId != null && quickStartUnitId != null) {
        val bitmap = getBitmapFromUri(context, uri)
        bitmap?.let {
            processQuickStartImage(
                viewModel = viewModel,
                bitmap = it,
                textbookId = quickStartTextbookId!!,
                unitId = quickStartUnitId!!,
                navController = navController
            )
        }
    }
}

Scaffold(
    topBar = {
        TopAppBar(
            title = { Text("英语音标点读") }
        )
    }
) { padding ->
    LazyColumn(
        modifier = Modifier
            .fillMaxSize()
    )
}

```

```

.padding(padding)
.padding(horizontal = 16.dp),
verticalArrangement = Arrangement.spacedBy(16.dp),
contentPadding = PaddingValues(vertical = 16.dp)
) {
    item {
        Column {
            Text(
                text = "英语音标点读",
                fontSize = 28.sp,
                fontWeight = FontWeight.Bold
            )
            Text(
                text = "给课本拍张照，秒变会说话的音标点读机",
                fontSize = 14.sp,
                color = MaterialTheme.colorScheme.onSurfaceVariant
            )
        }
    }
}

// 示例体验入口 —— 新用户一秒看到效果
if (demoTextbook != null && demoUnit != null && demoPage != null) {
    item {
        DemoEntryCard(
            textbook = demoTextbook,
            unit = demoUnit,
            page = demoPage,
            onClick = {
                navController.navigate("reader/${demoTextbook.id}/${demoUnit.id}/${demoPage.id}")
            }
        )
    }
}

// 快速开始按钮 —— 一键拍照直达点读
item {
    QuickStartButton(onClick = { showQuickStartSheet = true })
}

item {
    Row(
        modifier = Modifier.fillMaxWidth(),
        horizontalArrangement = Arrangement.SpaceBetween,
        verticalAlignment = Alignment.CenterVertically
    ) {
        Text(
            text = "  我的课本",
            fontSize = 18.sp,
            fontWeight = FontWeight.SemiBold
        )
        IconButton(onClick = { showAddDialog = true }) {
            Icon(
                imageVector = Icons.Default.AddCircle,

```

[illegible]

```

quickStartUnitId = it.second
})
) {
showQuickStartSheet = false
val file = File(context.cacheDir, "camera_${System.currentTimeMillis()}.jpg")
tempImageUri = FileProvider.getUriForFile(
context,
"${context.packageName}.fileprovider",
file
)
tempImageUri?.let { cameraLauncher.launch(it) }
},
modifier = Modifier.fillMaxWidth(),
shape = RoundedCornerShape(16.dp)
) {
Icon(Icons.Default.PhotoCamera, contentDescription = null)
Spacer(modifier = Modifier.width(8.dp))
Text(" 拍照", fontSize = 18.sp)
}
Button(
onClick = {
if (handleQuickStart(viewModel, textbooks, {
quickStartTextbookId = it.first
quickStartUnitId = it.second
}))
) {
showQuickStartSheet = false
photoPicker.launch(
PickVisualMediaRequest(ActivityResultContracts.PickVisualMedia.ImageOnly)
)
},
modifier = Modifier.fillMaxWidth(),
shape = RoundedCornerShape(16.dp),
colors = androidx.compose.material3.ButtonDefaults.buttonColors(
containerColor = MaterialTheme.colorScheme.surfaceVariant,
contentColor = MaterialTheme.colorScheme.onSurface
)
) {
Icon(Icons.Default.PhotoLibrary, contentDescription = null)
Spacer(modifier = Modifier.width(8.dp))
Text(" 从相册选择", fontSize = 18.sp)
}
Spacer(modifier = Modifier.height(8.dp))
}
}
}
if (showAddDialog) {
AlertDialog(

```

```

onDismissRequest = {
showAddDialog = false
newTextbookName = ""
},
title = { Text("添加课本") },
text = {
Column {
Text("请输入课本名称，例如： 人教版三年级上册")
Spacer(modifier = Modifier.height(8.dp))
OutlinedTextField(
value = newTextbookName,
onValueChange = { newTextbookName = it },
label = { Text("课本名称") },
singleLine = true
)
}
},
confirmButton = {
TextButton(
onClick = {
if (newTextbookName.isNotEmpty()) {
viewModel.addTextbook(newTextbookName)
newTextbookName = ""
showAddDialog = false
}
}
) {
Text("添加")
}
},
dismissButton = {
TextButton(onClick = {
showAddDialog = false
newTextbookName = ""
}) {
Text("取消")
}
}
}
if (isProcessing) {
com.liudong.bookread.ui.components.ProcessingOverlay(message = processingProgress)
}
}

// MARK: - 快速开始逻辑
/// 自动创建"我的课本"和"Unit 1"（如果还没有），返回 (textbookId, unitId)
private fun handleQuickStart(
viewModel: TextbookViewModel,
textbooks: List<Textbook>,
onResult: (Pair<String, String>) -> Unit

```

```

): Boolean {
    val userTextbooks = textbooks.filter { it.name != "示例课本" }
    return if (userTextbooks.isEmpty()) {
        val newTextbook = viewModel.addTextbook("我的课本")
        val unit = viewModel.addUnit(newTextbook.id, "Unit 1") ?: return false
        onResult(newTextbook.id to unit.id)
        true
    } else {
        val existing = userTextbooks.first()
        val unit = existing.units.firstOrNull()
        ?: viewModel.addUnit(existing.id, "Unit 1")
        ?: return false
        onResult(existing.id to unit.id)
        true
    }
}

/// 处理快速开始拍摄/选择的图片：添加到页面并直达点读页
private fun processQuickStartImage(
    viewModel: TextbookViewModel,
    bitmap: Bitmap,
    textbookId: String,
    unitId: String,
    navController: NavController
) {
    val totalPages = viewModel.textbooks.value
        .find { it.id == textbookId }?.units?.sumOf { it.pages.size } ?: 0
    val pageName = "Page ${totalPages + 1}"
    viewModel.addPage(textbookId, unitId, bitmap, pageName) { page ->
        if (page != null) {
            navController.navigate("reader/$textbookId/$unitId/${page.id}")
        }
    }
}

// MARK: - Demo 体验卡片
/// 新用户首次打开 App 时展示的 demo 入口，点击直接进入标注好的点读页
@Composable
fun DemoEntryCard(
    textbook: Textbook,
    unit: TextbookUnit,
    page: TextbookPage,
    onClick: () -> Unit
) {
    val context = LocalContext.current
    val bitmap = remember(page.imagePath) {
        DataStoreService.loadImage(context, page.imagePath)
    }

    val annotationCount = page.annotations.size
    Row(
        modifier = Modifier
            .fillMaxWidth()

```

```

.clip(RoundedCornerShape(14.dp))
.background(Color(0x0F2196F3))
.border(1.dp, Color(0x332196F3), RoundedCornerShape(14.dp))
.clickable(onClick = onClick)
.padding(vertical = 14.dp)
.padding(end = 10.dp),
verticalAlignment = Alignment.CenterVertically
) {
// 左侧文字区
Column(
modifier = Modifier
.weight(1f)
.padding(start = 16.dp),
verticalArrangement = Arrangement.spacedBy(6.dp)
) {
Row(verticalAlignment = Alignment.CenterVertically) {
Text(text = " ", fontSize = 20.sp)
Spacer(modifier = Modifier.width(6.dp))
Text(
text = "先看看效果",
fontSize = 17.sp,
fontWeight = FontWeight.SemiBold,
color = MaterialTheme.colorScheme.onSurface
)
}
Text(
text = "这是一张标注好的课本页，点击单词就能听到发音",
fontSize = 12.sp,
color = MaterialTheme.colorScheme.onSurfaceVariant,
maxLines = 2
)
if (annotationCount > 0) {
Row(verticalAlignment = Alignment.CenterVertically) {
Icon(
imageVector = Icons.Default.CheckCircle,
contentDescription = null,
tint = Color(0xFF4CAF50),
modifier = Modifier.size(14.dp)
)
Spacer(modifier = Modifier.width(4.dp))
Text(
text = "已识别 $annotationCount 个单词",
fontSize = 12.sp,
color = Color(0xFF4CAF50)
)
}
}
Row(verticalAlignment = Alignment.CenterVertically) {
Text(
text = "点击体验",

```

```

fontSize = 12.sp,
fontWeight = FontWeight.Medium,
color = Color(0xFF2196F3)
)
Icon(
imageVector = Icons.Default.KeyboardArrowRight,
contentDescription = null,
tint = Color(0xFF2196F3),
modifier = Modifier.size(14.dp)
)
}
}
// 右侧缩略图
Box(
modifier = Modifier
.size(width = 90.dp, height = 110.dp)
.clip(RoundedCornerShape(8.dp))
.background(Color(0x142196F3)),
contentAlignment = Alignment.Center
) {
if (bitmap != null) {
Image(
bitmap = bitmap.asImageBitmap(),
contentDescription = page.name,
modifier = Modifier.fillMaxSize(),
contentScale = ContentScale.Crop
)
} else {
Icon(
imageVector = Icons.Default.MenuBook,
contentDescription = null,
tint = Color(0x662196F3),
modifier = Modifier.size(32.dp)
)
}
}
}
// MARK: - 快速开始按钮
/// 一键启动: 自动创建课本+单元 → 拍照 → 直接进入点读页
@Composable
fun QuickStartButton(onClick: () -> Unit) {
Row(
modifier = Modifier
.fillMaxWidth()
.shadow(6.dp, RoundedCornerShape(14.dp))
.clip(RoundedCornerShape(14.dp))
.background(
Brush.horizontalGradient(
listOf(Color(0xFF2196F3), Color(0xFF42A5F5))

```

```

    )
    )
    .clickable(onClick = onClick)
    .padding(horizontal = 18.dp, vertical = 14.dp),
    verticalAlignment = Alignment.CenterVertically
  ) {
    Icon(
      imageVector = Icons.Default.PhotoCamera,
      contentDescription = null,
      tint = Color.White,
      modifier = Modifier.size(18.dp)
    )
    Spacer(modifier = Modifier.width(10.dp))
    Text(
      text = "  快速开始 — 拍照识别课本",
      fontSize = 15.sp,
      fontWeight = FontWeight.SemiBold,
      color = Color.White,
      modifier = Modifier.weight(1f)
    )
    Icon(
      imageVector = Icons.Default.ArrowForward,
      contentDescription = null,
      tint = Color.White,
      modifier = Modifier.size(20.dp)
    )
  }
}

@Composable
fun TextbookCard(
  textbook: Textbook,
  onClick: () -> Unit
) {
  val totalPages = textbook.units.sumOf { it.pages.size }
  Card(
    onClick = onClick,
    shape = RoundedCornerShape(16.dp),
    colors = CardDefaults.cardColors(containerColor = Color.White),
    elevation = CardDefaults.cardElevation(defaultElevation = 2.dp),
    modifier = Modifier.fillMaxWidth()
  ) {
    Row(
      modifier = Modifier
        .fillMaxWidth()
        .padding(16.dp),
      verticalAlignment = Alignment.CenterVertically
    ) {
      Box(
        modifier = Modifier
          .size(width = 70.dp, height = 90.dp)

```

```

        .background(Color(0xFFE8F5E9), RoundedCornerShape(12.dp)),
        contentAlignment = Alignment.Center
    ) {
        Icon(
            imageVector = Icons.Default.MenuBook,
            contentDescription = null,
            tint = Color(0xFF4CAF50),
            modifier = Modifier.size(32.dp)
        )
    }
    Column(
        modifier = Modifier
            .weight(1f)
            .padding(start = 16.dp)
    ) {
        Text(
            text = textbook.name,
            fontSize = 16.sp,
            fontWeight = FontWeight.SemiBold,
            maxLines = 2
        )
        Text(
            text = "$totalPages 页 · ${textbook.units.size} 个单元",
            fontSize = 14.sp,
            color = MaterialTheme.colorScheme.onSurfaceVariant
        )
        Text(
            text = "最近学习: ${formatDate(textbook.createdAt)}",
            fontSize = 12.sp,
            color = Color.Gray
        )
    }
}

@Composable
fun EmptyTextbooksView() {
    Column(
        modifier = Modifier
            .fillMaxWidth()
            .padding(vertical = 40.dp),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.spacedBy(12.dp)
    ) {
        Icon(
            imageVector = Icons.Default.CameraAlt,
            contentDescription = null,
            tint = Color.Gray.copy(alpha = 0.5f),
            modifier = Modifier.size(60.dp)
        )
    }
}

```

```

Text(
    text = "还没有课本",
    fontSize = 16.sp,
    fontWeight = FontWeight.Medium,
    color = Color.Gray
)
Text(
    text = "点击右上角 + 号，添加孩子的英语课本",
    fontSize = 14.sp,
    color = Color.Gray
)
}
}

private fun formatDate(timestamp: Long): String {
    val sdf = SimpleDateFormat("yyyy-MM-dd", Locale.getDefault())
    return sdf.format(Date(timestamp))
}

// File: app/src/main/java/com/liudong/bookread/ui/detail/TextbookDetailScreen.kt
package com.liudong.bookread.ui.detail

import android.graphics.Bitmap
import android.net.Uri
import androidx.activity.compose.rememberLauncherForActivityResult
import androidx.activity.result.PickVisualMediaRequest
import androidx.activity.result.contract.ActivityResultContracts
import androidx.compose.animation.AnimatedVisibility
import androidx.compose.foundation.Image
import androidx.compose.foundation.background
import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.ExperimentalLayoutApi
import androidx.compose.foundation.layout.FlowRow
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.verticalScroll
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Add
import androidx.compose.material.icons.filled.ArrowDropDown
import androidx.compose.material.icons.filled.KeyboardArrowUp
import androidx.compose.material.icons.filled.MenuBook
import androidx.compose.material.icons.filled.MoreVert

```

```

import androidx.compose.material.icons.filled.PhotoCamera
import androidx.compose.material.icons.filled.PhotoLibrary
import androidx.compose.material3.AlertDialog
import androidx.compose.material3.Button
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.DropdownMenu
import androidx.compose.material3.DropdownMenuItem
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.ModalBottomSheet
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Text
import androidx.compose.material3.TextButton
import androidx.compose.material3.TopAppBar
import androidx.compose.material3.rememberModalBottomSheetState
import androidx.compose.runtime.Composable
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.asImageBitmap
import androidx.compose.ui.layout.ContentScale
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.core.content.FileProvider
import androidx.navigation.NavController
import com.liudong.bookread.model.Textbook
import com.liudong.bookread.model.TextbookPage
import com.liudong.bookread.model.TextbookUnit
import com.liudong.bookread.service.DataStoreService
import com.liudong.bookread.viewmodel.TextbookViewModel
import java.io.File

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun TextbookDetailScreen(
    viewModel: TextbookViewModel,
    textbookId: String,
    navController: NavController
) {

```

```

val textbooks by viewModel.textbooks.collectAsState()
val textbook = textbooks.find { it.id == textbookId } ?: return
val isProcessing by viewModel.isProcessing.collectAsState()
val processingProgress by viewModel.processingProgress.collectAsState()
val context = LocalContext.current
var showAddUnit by remember { mutableStateOf(false) }
var newUnitName by remember { mutableStateOf("") }
var showImagePicker by remember { mutableStateOf(false) }
var selectedUnitId by remember { mutableStateOf<String?>(null) }
var showRenameDialog by remember { mutableStateOf(false) }
var pageToRename by remember { mutableStateOf<TextbookPage?>(null) }
var unitIdForRename by remember { mutableStateOf<String?>(null) }
var newPageName by remember { mutableStateOf("") }
var tempImageUri by remember { mutableStateOf<Uri?>(null) }
val cameraLauncher = rememberLauncherForActivityResult(
    ActivityResultContracts.TakePicture()
) { success ->
    if (success && tempImageUri != null && selectedUnitId != null) {
        val bitmap = com.liudong.bookread.ui.components.getBitmapFromUri(context, tempImageUri!!)
        bitmap?.let {
            addPage(viewModel, textbookId, selectedUnitId!!, it, textbook)
        }
        tempImageUri = null
    }
}

val photoPicker = rememberLauncherForActivityResult(
    ActivityResultContracts.PickVisualMedia()
) { uri ->
    if (uri != null && selectedUnitId != null) {
        val bitmap = com.liudong.bookread.ui.components.getBitmapFromUri(context, uri)
        bitmap?.let {
            addPage(viewModel, textbookId, selectedUnitId!!, it, textbook)
        }
    }
}

Scaffold(
    topBar = {
        TopAppBar(
            title = { Text("课本详情") }
        )
    }
) { padding ->
    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(padding)
            .verticalScroll(rememberScrollState())
            .padding(16.dp),
        verticalArrangement = Arrangement.spacedBy(16.dp)
    ) {

```

```

Row(
  verticalAlignment = Alignment.CenterVertically,
  horizontalArrangement = Arrangement.spacedBy(16.dp)
) {
  Box(
    modifier = Modifier
      .size(width = 80.dp, height = 100.dp)
      .background(Color(0xFFE8F5E9), RoundedCornerShape(12.dp)),
    contentAlignment = Alignment.Center
  ) {
    Icon(
      imageVector = Icons.Default.MenuBook,
      contentDescription = null,
      tint = Color(0xFF4CAF50),
      modifier = Modifier.size(40.dp)
    )
  }
  Column {
    Text(
      text = textbook.name,
      fontSize = 22.sp,
      fontWeight = FontWeight.Bold
    )
    Text(
      text = "${textbook.units.sumOf { it.pages.size }} 页 · ${textbook.units.size} 个单元",
      fontSize = 14.sp,
      color = MaterialTheme.colorScheme.onSurfaceVariant
    )
  }
  Button(
    onClick = { showAddUnit = true },
    shape = RoundedCornerShape(20.dp)
  ) {
    Icon(Icons.Default.Add, contentDescription = null)
    Spacer(modifier = Modifier.width(4.dp))
    Text("添加单元")
  }
  if (textbook.units.isEmpty()) {
    EmptyUnitView()
  } else {
    textbook.units.sortedBy { it.order }.forEach { unit ->
      UnitSection(
        viewModel = viewModel,
        textbookId = textbookId,
        unit = unit,
        onAddPage = {
          selectedUnitId = unit.id
          showImagePicker = true
        },
        onRenamePage = { page, uId ->

```

```

"meaning": "侵蚀",
"level": "junior"
},
"contamination": {
"phonetic": "/kən,tæmɪ'neɪʃn/",
"meaning": "污染",
"level": "junior"
},
"physical": {
"phonetic": "/'fɪzɪkl/",
"meaning": "身体的",
"level": "junior"
},
"mental": {
"phonetic": "/'mentl/",
"meaning": "心理的",
"level": "junior"
},
"psychological": {
"phonetic": "/,saɪkə'lɒdʒɪkl/",
"meaning": "心理学的",
"level": "junior"
},
"therapy": {
"phonetic": "/'θ erəpi/",
"meaning": "治疗",
"level": "junior"
},
"cure": {
"phonetic": "/kjʊə(r)/",
"meaning": "治愈",
"level": "junior"
},
"heal": {
"phonetic": "/hi:l/",
"meaning": "愈合",
"level": "junior"
},
"relief": {
"phonetic": "/rɪ'li:f/",
"meaning": "缓解",
"level": "junior"
},
"symptom": {
"phonetic": "/'sɪmptəm/",
"meaning": "症状",
"level": "junior"
},
"diagnosis": {

```

```

"phonetic": "/,daɪəg'nəʊsɪs/",
"meaning": "诊断",
"level": "junior"
},
"prescription": {
"phonetic": "/prɪ'skrɪpʃn/",
"meaning": "处方",
"level": "junior"
},
"pharmacy": {
"phonetic": "/'fɑ:məsi/",
"meaning": "药房",
"level": "junior"
},
"surgeon": {
"phonetic": "/'sɜ:dʒən/",
"meaning": "外科医生",
"level": "junior"
},
"physician": {
"phonetic": "/fɪ'zɪʃn/",
"meaning": "内科医生",
"level": "junior"
},
"specialist": {
"phonetic": "/'speʃəlɪst/",
"meaning": "专家",
"level": "junior"
},
"bacteria": {
"phonetic": "/bæk'tɪəriə/",
"meaning": "细菌",
"level": "junior"
},
"gene": {
"phonetic": "/dʒi:n/",
"meaning": "基因",
"level": "junior"
},
"academic": {
"phonetic": "/,ækə'demɪk/",
"meaning": "学术的",
"level": "junior"
},
"scholar": {
"phonetic": "/'skɒlə(r)/",
"meaning": "学者",
"level": "junior"
},
"thesis": {

```

```

"phonetic": "/ˈθ iːsɪs/",
"meaning": "论文",
"level": "junior"
},
"dissertation": {
"phonetic": "/ˌdɪsəˈteɪʃn/",
"meaning": "学位论文",
"level": "junior"
},
"discipline": {
"phonetic": "/ˈdɪsəplɪn/",
"meaning": "学科",
"level": "junior"
},
"syllabus": {
"phonetic": "/ˈsɪləbəs/",
"meaning": "教学大纲",
"level": "junior"
},
"tuition": {
"phonetic": "/tjuˈɪʃn/",
"meaning": "学费",
"level": "junior"
},
"literature": {
"phonetic": "/ˈlɪtrətʃə(r)/",
"meaning": "文学",
"level": "junior"
},
"nonfiction": {
"phonetic": "/nɒnˈfɪkʃn/",
"meaning": "非虚构",
"level": "junior"
},
"prose": {
"phonetic": "/prəʊz/",
"meaning": "散文",
"level": "junior"
},
"drama": {
"phonetic": "/ˈdrɑ:mə/",
"meaning": "戏剧",
"level": "junior"
},
"comedy": {
"phonetic": "/ˈkɒmədi/",
"meaning": "喜剧",
"level": "junior"
},
"tragedy": {

```

```

"phonetic": "/ˈtrædʒədi/",
"meaning": "悲剧",
"level": "junior"
},
"plot": {
"phonetic": "/plot/",
"meaning": "情节",
"level": "junior"
},
"theme": {
"phonetic": "/θ i:m/",
"meaning": "主题",
"level": "junior"
},
"style": {
"phonetic": "/stail/",
"meaning": "风格",
"level": "junior"
},
"masterpiece": {
"phonetic": "/ˈmɑ:stəpi:s/",
"meaning": "杰作",
"level": "junior"
},
"mass media": {
"phonetic": "/mæs ˈmi:diə/",
"meaning": "大众媒体",
"level": "junior"
},
"social media": {
"phonetic": "/ˈsəʊʃl ˈmi:diə/",
"meaning": "社交媒体",
"level": "junior"
},
"journalism": {
"phonetic": "/ˈdʒɜ:nəlɪzəm/",
"meaning": "新闻业",
"level": "junior"
},
"reporter": {
"phonetic": "/rɪˈpɔ:tə(r)/",
"meaning": "记者",
"level": "junior"
},
"editor": {
"phonetic": "/ˈeditə(r)/",
"meaning": "编辑",
"level": "junior"
},
"publisher": {

```

```

"phonetic": "'pʌblɪʃə(r)"/,
"meaning": "出版商",
"level": "junior"
},
"viewer": {
"phonetic": "'vju:ə(r)"/,
"meaning": "观众",
"level": "junior"
},
"listener": {
"phonetic": "'lɪsənə(r)"/,
"meaning": "听众",
"level": "junior"
},
"reader": {
"phonetic": "'ri:də(r)"/,
"meaning": "读者",
"level": "junior"
},
"critic": {
"phonetic": "'krɪtɪk"/,
"meaning": "评论家",
"level": "junior"
},
"historical": {
"phonetic": "/hɪ'stɒrɪkl"/,
"meaning": "历史的",
"level": "junior"
},
"medieval": {
"phonetic": "/,medi'i:vəl"/,
"meaning": "中世纪的",
"level": "junior"
},
"contemporary": {
"phonetic": "/kən'tempərəi"/,
"meaning": "当代的",
"level": "junior"
},
"prehistoric": {
"phonetic": "/,pri:hi'stɒrɪk"/,
"meaning": "史前的",
"level": "junior"
},
"civilization": {
"phonetic": "/,sɪvəlaɪ'zeɪʃn"/,
"meaning": "文明",
"level": "junior"
},
"dynasty": {

```

```

"phonetic": "'dɪnəsti/",
"meaning": "朝代",
"level": "junior"
},
"colony": {
"phonetic": "'kɒləni/",
"meaning": "殖民地",
"level": "junior"
},
"independence": {
"phonetic": "/,ɪndɪ'pendəns/",
"meaning": "独立",
"level": "junior"
},
"war": {
"phonetic": "/wɔ:(r)/",
"meaning": "战争",
"level": "junior"
},
"philosophy": {
"phonetic": "/fɪ'lɒsəfi/",
"meaning": "哲学",
"level": "junior"
},
"philosopher": {
"phonetic": "/fɪ'lɒsəfə(r)/",
"meaning": "哲学家",
"level": "junior"
},
"religious": {
"phonetic": "/rɪ'lɪdʒəs/",
"meaning": "宗教的",
"level": "junior"
},
"spiritual": {
"phonetic": "'spɪrɪtʃuəl/",
"meaning": "精神的",
"level": "junior"
},
"soul": {
"phonetic": "/səʊl/",
"meaning": "灵魂",
"level": "junior"
},
"spirit": {
"phonetic": "'spɪrɪt/",
"meaning": "精神",
"level": "junior"
},
"thinking": {

```

```

"phonetic": "/ˈθ ɪŋkɪŋ/",
"meaning": "思考",
"level": "junior"
},
"ideology": {
"phonetic": "/,aɪdɪˈɒlədʒi/",
"meaning": "意识形态",
"level": "junior"
},
"moral": {
"phonetic": "/ˈmɒrəl/",
"meaning": "道德的",
"level": "junior"
},
"ethics": {
"phonetic": "/ˈeθ ɪks/",
"meaning": "伦理学",
"level": "junior"
},
"virtue": {
"phonetic": "/ˈvɜ:tʃu:/",
"meaning": "美德",
"level": "junior"
},
"sin": {
"phonetic": "/sɪn/",
"meaning": "罪恶",
"level": "junior"
},
"mathematics": {
"phonetic": "/,mæθ əˈmætɪks/",
"meaning": "数学",
"level": "junior"
},
"formula": {
"phonetic": "/ˈfɔ:mjələ/",
"meaning": "公式",
"level": "junior"
},
"equation": {
"phonetic": "/ɪˈkweɪʒn/",
"meaning": "方程",
"level": "junior"
},
"function": {
"phonetic": "/ˈfʌŋkʃn/",
"meaning": "函数",
"level": "junior"
},
"variable": {

```

```

"phonetic": "'veəriəbl/",
"meaning": "变量",
"level": "junior"
},
"constant": {
"phonetic": "'konstənt/",
"meaning": "常数",
"level": "junior"
},
"calculate": {
"phonetic": "'kælkjuleɪt/",
"meaning": "计算",
"level": "junior"
},
"compute": {
"phonetic": "'kəm'pjʊ:t/",
"meaning": "计算",
"level": "junior"
},
"measure": {
"phonetic": "'meɪzə(r)/",
"meaning": "测量",
"level": "junior"
},
"estimate": {
"phonetic": "'estɪmeɪt/",
"meaning": "估计",
"level": "junior"
},
"statistics": {
"phonetic": "'stætɪstɪks/",
"meaning": "统计学",
"level": "junior"
},
"probability": {
"phonetic": "'prɒbə'bɪləti/",
"meaning": "概率",
"level": "junior"
},
"ratio": {
"phonetic": "'reɪʃiə/",
"meaning": "比率",
"level": "junior"
},
"proportion": {
"phonetic": "'prɒ'pɔ:ʃn/",
"meaning": "比例",
"level": "junior"
},
"physics": {

```

```

"phonetic": "/ˈfɪzɪks/",
"meaning": "物理",
"level": "junior"
},
"chemistry": {
"phonetic": "/ˈkɛmɪstri/",
"meaning": "化学",
"level": "junior"
},
"biology": {
"phonetic": "/baɪˈɒlədʒi/",
"meaning": "生物",
"level": "junior"
},
"geography": {
"phonetic": "/dʒɪˈɒɡrəfi/",
"meaning": "地理",
"level": "junior"
},
"geology": {
"phonetic": "/dʒɪˈɒlədʒi/",
"meaning": "地质学",
"level": "junior"
},
"astronomy": {
"phonetic": "/əˈstrɒnəmi/",
"meaning": "天文学",
"level": "junior"
},
"atom": {
"phonetic": "/ˈætəm/",
"meaning": "原子",
"level": "junior"
},
"molecule": {
"phonetic": "/ˈmɒlɪkjʊl/",
"meaning": "分子",
"level": "junior"
},
"particle": {
"phonetic": "/ˈpɑːtɪkl/",
"meaning": "粒子",
"level": "junior"
},
"electron": {
"phonetic": "/ɪˈlektɹɒn/",
"meaning": "电子",
"level": "junior"
},
"magnet": {

```

```

"phonetic": "'mæɡnɪt/",
"meaning": "磁铁",
"level": "junior"
},
"gravity": {
"phonetic": "'grævəti/",
"meaning": "重力",
"level": "junior"
},
"friction": {
"phonetic": "'frɪkʃn/",
"meaning": "摩擦力",
"level": "junior"
},
"volume": {
"phonetic": "'vɒlju:m/",
"meaning": "体积",
"level": "junior"
},
"continent": {
"phonetic": "'kɒntɪnənt/",
"meaning": "大陆",
"level": "junior"
},
"plateau": {
"phonetic": "'plætəʊ/",
"meaning": "高原",
"level": "junior"
},
"plain": {
"phonetic": "/pleɪn/",
"meaning": "平原",
"level": "junior"
},
"basin": {
"phonetic": "'beɪsn/",
"meaning": "盆地",
"level": "junior"
},
"canyon": {
"phonetic": "'kænjən/",
"meaning": "峡谷",
"level": "junior"
},
"glacier": {
"phonetic": "'glæsiə(r)/",
"meaning": "冰川",
"level": "junior"
},
"volcano": {

```

```

"phonetic": "/vɒl'keɪnəθ/",
"meaning": "火山",
"level": "junior"
},
"tsunami": {
"phonetic": "/tsu:'nɑ:mi/",
"meaning": "海啸",
"level": "junior"
},
"tide": {
"phonetic": "/taɪd/",
"meaning": "潮汐",
"level": "junior"
},
"current": {
"phonetic": "/'kʌrənt/",
"meaning": "洋流",
"level": "junior"
},
"latitude": {
"phonetic": "/'lætɪtju:d/",
"meaning": "纬度",
"level": "junior"
},
"longitude": {
"phonetic": "/'lɒŋɡɪtju:d/",
"meaning": "经度",
"level": "junior"
},
"altitude": {
"phonetic": "/'æltɪtju:d/",
"meaning": "海拔",
"level": "junior"
},
"military": {
"phonetic": "/'mɪlətri/",
"meaning": "军事的",
"level": "junior"
},
"navy": {
"phonetic": "/'neɪvi/",
"meaning": "海军",
"level": "junior"
},
"air force": {
"phonetic": "/eə fɔ:s/",
"meaning": "空军",
"level": "junior"
},
"weapon": {

```

```

"phonetic": "'wepən/",
"meaning": "武器",
"level": "junior"
},
"defense": {
"phonetic": "/di'fens/",
"meaning": "防御",
"level": "junior"
},
"safety": {
"phonetic": "'seifti/",
"meaning": "安全",
"level": "junior"
},
"danger": {
"phonetic": "'deɪndʒə(r)/",
"meaning": "危险",
"level": "junior"
},
"crime": {
"phonetic": "/kraɪm/",
"meaning": "犯罪",
"level": "junior"
},
"criminal": {
"phonetic": "'krɪmɪnl/",
"meaning": "罪犯",
"level": "junior"
},
"theft": {
"phonetic": "/θ eft/",
"meaning": "盗窃",
"level": "junior"
},
"robbery": {
"phonetic": "'rɒbəri/",
"meaning": "抢劫",
"level": "junior"
},
"murder": {
"phonetic": "'mɜ:də(r)/",
"meaning": "谋杀",
"level": "junior"
},
"violence": {
"phonetic": "'vaɪələns/",
"meaning": "暴力",
"level": "junior"
},
"victim": {

```

```

"phonetic": "'vɪktɪm/",
"meaning": "受害者",
"level": "junior"
},
"witness": {
"phonetic": "'wɪtnəs/",
"meaning": "证人",
"level": "junior"
},
"evidence": {
"phonetic": "'evidəns/",
"meaning": "证据",
"level": "junior"
},
"trial": {
"phonetic": "'traɪəl/",
"meaning": "审判",
"level": "junior"
},
"architecture": {
"phonetic": "'ɑ:kɪtektʃə(r)/",
"meaning": "建筑",
"level": "junior"
},
"architect": {
"phonetic": "'ɑ:kɪtekt/",
"meaning": "建筑师",
"level": "junior"
},
"structure": {
"phonetic": "'strʌktʃə(r)/",
"meaning": "结构",
"level": "junior"
},
"construct": {
"phonetic": "'kɒn'strʌkt/",
"meaning": "建造",
"level": "junior"
},
"demolish": {
"phonetic": "'di'mɒlɪʃ/",
"meaning": "拆除",
"level": "junior"
},
"foundation": {
"phonetic": "'faʊn'deɪʃn/",
"meaning": "基础",
"level": "junior"
},
"concrete": {

```

```

"phonetic": "'kɒŋkri:t/",
"meaning": "混凝土",
"level": "junior"
},
"steel": {
"phonetic": "/sti:l/",
"meaning": "钢",
"level": "junior"
},
"transport": {
"phonetic": "'trænsɒ:t/",
"meaning": "运输",
"level": "junior"
},
"traffic": {
"phonetic": "'træfɪk/",
"meaning": "交通",
"level": "junior"
},
"vehicle": {
"phonetic": "'vi:əkl/",
"meaning": "车辆",
"level": "junior"
},
"passenger": {
"phonetic": "'pæsɪndʒə(r)/",
"meaning": "乘客",
"level": "junior"
},
"pedestrian": {
"phonetic": "/pə'destrian/",
"meaning": "行人",
"level": "junior"
},
"license": {
"phonetic": "'ləɪsɪns/",
"meaning": "执照",
"level": "junior"
},
"fuel": {
"phonetic": "'fju:əl/",
"meaning": "燃料",
"level": "junior"
},
"petrol": {
"phonetic": "'petrəl/",
"meaning": "汽油",
"level": "junior"
},
"gasoline": {

```

```

"phonetic": "'gæsəli:n/",
"meaning": "汽油",
"level": "junior"
},
"farming": {
"phonetic": "'fa:miŋ/",
"meaning": "农耕",
"level": "junior"
},
"crop": {
"phonetic": "/krop/",
"meaning": "庄稼",
"level": "junior"
},
"harvest": {
"phonetic": "'hɑ:vɪst/",
"meaning": "收获",
"level": "junior"
},
"grain": {
"phonetic": "/greɪn/",
"meaning": "谷物",
"level": "junior"
},
"wheat": {
"phonetic": "/wi:t/",
"meaning": "小麦",
"level": "junior"
},
"fertilizer": {
"phonetic": "'fɜ:təlaɪzə(r)/",
"meaning": "肥料",
"level": "junior"
},
"pesticide": {
"phonetic": "'pestɪsaɪd/",
"meaning": "杀虫剂",
"level": "junior"
},
"irrigation": {
"phonetic": "/,ɪrɪ'geɪʃn/",
"meaning": "灌溉",
"level": "junior"
},
"livestock": {
"phonetic": "'laɪvstɒk/",
"meaning": "牲畜",
"level": "junior"
},
"artistic": {

```

```

"phonetic": "/ɑ:'tɪstɪk/",
"meaning": "艺术的",
"level": "junior"
},
"gallery": {
"phonetic": "'gæləri/",
"meaning": "画廊",
"level": "junior"
},
"sculpture": {
"phonetic": "'skʌlptʃə(r)/",
"meaning": "雕塑",
"level": "junior"
},
"statue": {
"phonetic": "'stætʃu:/",
"meaning": "雕像",
"level": "junior"
},
"portrait": {
"phonetic": "'pɔ:treɪt/",
"meaning": "肖像",
"level": "junior"
},
"musical": {
"phonetic": "'mju:zɪkl/",
"meaning": "音乐的",
"level": "junior"
},
"instrument": {
"phonetic": "'ɪnstɹəmənt/",
"meaning": "乐器",
"level": "junior"
},
"orchestra": {
"phonetic": "'ɔ:kɪstrə/",
"meaning": "管弦乐队",
"level": "junior"
},
"concert": {
"phonetic": "'kɒnsət/",
"meaning": "音乐会",
"level": "junior"
},
"performance": {
"phonetic": "pə'fɔ:məns/",
"meaning": "表演",
"level": "junior"
},
"rehearsal": {

```

```

"phonetic": "/rɪ'hɜːsl/",
"meaning": "排练",
"level": "junior"
},
"stage": {
"phonetic": "/steɪdʒ/",
"meaning": "阶段",
"level": "junior"
},
"athletic": {
"phonetic": "/æθ 'letɪk/",
"meaning": "运动的",
"level": "junior"
},
"tournament": {
"phonetic": "/'tɔənəmənt/",
"meaning": "锦标赛",
"level": "junior"
},
"league": {
"phonetic": "/li:g/",
"meaning": "联盟",
"level": "junior"
},
"referee": {
"phonetic": "/,refə'ri:/",
"meaning": "裁判",
"level": "junior"
},
"training": {
"phonetic": "/'treɪnɪŋ/",
"meaning": "训练",
"level": "junior"
},
"characteristic": {
"phonetic": "/,kærəktə'rɪstɪk/",
"meaning": "特征",
"level": "junior"
},
"feature": {
"phonetic": "/'fi:tʃə(r)/",
"meaning": "特点",
"level": "junior"
},
"trait": {
"phonetic": "/treɪt/",
"meaning": "特性",
"level": "junior"
},
"attribute": {

```

```

"phonetic": "/ˈætrɪbjʊ:t/",
"meaning": "属性",
"level": "junior"
},
"property": {
"phonetic": "/ˈprɒpəti/",
"meaning": "性质",
"level": "junior"
},
"aspect": {
"phonetic": "/ˈæspekt/",
"meaning": "方面",
"level": "junior"
},
"element": {
"phonetic": "/ˈelɪmənt/",
"meaning": "元素",
"level": "junior"
},
"factor": {
"phonetic": "/ˈfæktə(r)/",
"meaning": "因素",
"level": "junior"
},
"component": {
"phonetic": "/kəmˈpəʊnənt/",
"meaning": "组成部分",
"level": "junior"
},
"ingredient": {
"phonetic": "/ɪnˈɡriːdiənt/",
"meaning": "成分",
"level": "junior"
},
"substance": {
"phonetic": "/ˈsʌbstəns/",
"meaning": "物质",
"level": "junior"
},
"essence": {
"phonetic": "/ˈesns/",
"meaning": "本质",
"level": "junior"
},
"process": {
"phonetic": "/ˈprəʊses/",
"meaning": "过程",
"level": "junior"
},
"procedure": {

```

```

"phonetic": "/prə'si:dʒə(r)/",
"meaning": "程序",
"level": "junior"
},
"phase": {
"phonetic": "/feɪz/",
"meaning": "阶段",
"level": "junior"
},
"period": {
"phonetic": "/ˈpɪəriəd/",
"meaning": "时期",
"level": "junior"
},
"era": {
"phonetic": "/ˈɪərə/",
"meaning": "时代",
"level": "junior"
},
"decade": {
"phonetic": "/ˈdekeɪd/",
"meaning": "十年",
"level": "junior"
},
"millennium": {
"phonetic": "/mɪˈleniəm/",
"meaning": "千年",
"level": "junior"
},
"origin": {
"phonetic": "/ˈɒrɪdʒɪn/",
"meaning": "起源",
"level": "junior"
},
"source": {
"phonetic": "/sɔ:s/",
"meaning": "来源",
"level": "junior"
},
"root": {
"phonetic": "/ru:t/",
"meaning": "根源",
"level": "junior"
},
"outcome": {
"phonetic": "/ˈaʊtkʌm/",
"meaning": "结果",
"level": "junior"
},
"consequence": {

```

```

"phonetic": "'kɒnsɪkwəns/",
"meaning": "后果",
"level": "junior"
},
"impact": {
"phonetic": "'ɪmpækt/",
"meaning": "冲击",
"level": "junior"
},
"affect": {
"phonetic": "'ə'fekt/",
"meaning": "影响",
"level": "junior"
},
"summary": {
"phonetic": "'sʌməri/",
"meaning": "总结",
"level": "junior"
},
"finding": {
"phonetic": "'faɪndɪŋ/",
"meaning": "发现",
"level": "junior"
},
"phenomenon": {
"phonetic": "'fə'nɒmɪnən/",
"meaning": "现象",
"level": "junior"
},
"trend": {
"phonetic": "/trend/",
"meaning": "趋势",
"level": "junior"
},
"pattern": {
"phonetic": "'pætn/",
"meaning": "模式",
"level": "junior"
},
"tendency": {
"phonetic": "'tendənsi/",
"meaning": "倾向",
"level": "junior"
},
"extent": {
"phonetic": "/ɪk'stent/",
"meaning": "程度",
"level": "junior"
},
"scale": {

```

```

"phonetic": "/skeɪl/",
"meaning": "规模",
"level": "junior"
},
"scope": {
"phonetic": "/skəʊp/",
"meaning": "范围",
"level": "junior"
},
"range": {
"phonetic": "/reɪndʒ/",
"meaning": "范围",
"level": "junior"
},
"spectrum": {
"phonetic": "/ˈspektrəm/",
"meaning": "光谱",
"level": "junior"
},
"variety": {
"phonetic": "/vəˈraɪəti/",
"meaning": "多样性",
"level": "junior"
},
"diversity": {
"phonetic": "/daɪˈvɜːsəti/",
"meaning": "多样性",
"level": "junior"
},
"complexity": {
"phonetic": "/kəmˈpleksəti/",
"meaning": "复杂性",
"level": "junior"
},
"intensity": {
"phonetic": "/ɪnˈtensəti/",
"meaning": "强度",
"level": "junior"
},
"give up": {
"phonetic": "/ɡɪv ʌp/",
"meaning": "放弃",
"level": "junior"
},
"set up": {
"phonetic": "/set ʌp/",
"meaning": "建立",
"level": "junior"
},
"pick up": {

```

```

"phonetic": "/pɪk ʌp/",
"meaning": "捡起; 学会",
"level": "junior"
},
"bring up": {
"phonetic": "/brɪŋ ʌp/",
"meaning": "养育",
"level": "junior"
},
"grow up": {
"phonetic": "/grəʊ ʌp/",
"meaning": "长大",
"level": "junior"
},
"look up": {
"phonetic": "/lʊk ʌp/",
"meaning": "查阅",
"level": "junior"
},
"take up": {
"phonetic": "/teɪk ʌp/",
"meaning": "开始从事",
"level": "junior"
},
"make up": {
"phonetic": "/meɪk ʌp/",
"meaning": "编造; 化妆",
"level": "junior"
},
"break up": {
"phonetic": "/breɪk ʌp/",
"meaning": "分手; 解散",
"level": "junior"
},
"catch up": {
"phonetic": "/kætʃ ʌp/",
"meaning": "赶上",
"level": "junior"
},
"keep up": {
"phonetic": "/ki:p ʌp/",
"meaning": "保持",
"level": "junior"
},
"speed up": {
"phonetic": "/spi:d ʌp/",
"meaning": "加速",
"level": "junior"
},
"slow down": {

```

```

"phonetic": "/sləʊ daʊn/",
"meaning": "减速",
"level": "junior"
},
"calm down": {
"phonetic": "/kɑ:m daʊn/",
"meaning": "冷静下来",
"level": "junior"
},
"break down": {
"phonetic": "/breɪk daʊn/",
"meaning": "出故障",
"level": "junior"
},
"cut down": {
"phonetic": "/kʌt daʊn/",
"meaning": "削减",
"level": "junior"
},
"figure out": {
"phonetic": "/'fɪgə(r) aʊt/",
"meaning": "弄清楚",
"level": "junior"
},
"find out": {
"phonetic": "/faɪnd aʊt/",
"meaning": "发现",
"level": "junior"
},
"point out": {
"phonetic": "/pɔɪnt aʊt/",
"meaning": "指出",
"level": "junior"
},
"run out": {
"phonetic": "/rʌn aʊt/",
"meaning": "用完",
"level": "junior"
},
"significant": {
"phonetic": "/sɪg'nɪfɪkənt/",
"meaning": "重要的",
"level": "junior"
},
"essential": {
"phonetic": "/ɪ'senʃl/",
"meaning": "必不可少的",
"level": "junior"
},
"vital": {

```

```

"phonetic": "'vairtl/",
"meaning": "至关重要的",
"level": "junior"
},
"crucial": {
"phonetic": "'kru:ʃl/",
"meaning": "关键的",
"level": "junior"
},
"fundamental": {
"phonetic": "/,fʌndə'mentl/",
"meaning": "根本的",
"level": "junior"
},
"basic": {
"phonetic": "'beɪsɪk/",
"meaning": "基础的",
"level": "junior"
},
"primary": {
"phonetic": "'praɪməri/",
"meaning": "主要的",
"level": "junior"
},
"major": {
"phonetic": "'meɪdʒə(r)/",
"meaning": "主要的",
"level": "junior"
},
"minor": {
"phonetic": "'maɪnə(r)/",
"meaning": "次要的",
"level": "junior"
},
"acquire": {
"phonetic": "/ə'kwəɪə(r)/",
"meaning": "获得",
"level": "junior"
},
"obtain": {
"phonetic": "/əb'teɪn/",
"meaning": "得到",
"level": "junior"
},
"gain": {
"phonetic": "/geɪn/",
"meaning": "获得",
"level": "junior"
},
"earn": {

```

```

"phonetic": "/ɜ:n/",
"meaning": "赚取",
"level": "junior"
},
"deserve": {
"phonetic": "/di'zɜ:v/",
"meaning": "值得",
"level": "junior"
},
"belong": {
"phonetic": "/br'lon/",
"meaning": "属于",
"level": "junior"
},
"own": {
"phonetic": "/əʊn/",
"meaning": "拥有",
"level": "junior"
},
"possess": {
"phonetic": "/pə'zes/",
"meaning": "占有",
"level": "junior"
},
"owe": {
"phonetic": "/əʊ/",
"meaning": "欠",
"level": "junior"
},
"lack": {
"phonetic": "/læk/",
"meaning": "缺乏",
"level": "junior"
},
"contain": {
"phonetic": "/kən'tein/",
"meaning": "包含",
"level": "junior"
},
"include": {
"phonetic": "/in'klu:d/",
"meaning": "包括",
"level": "junior"
},
"exclude": {
"phonetic": "/ɪk'sklu:d/",
"meaning": "排除",
"level": "junior"
},
"involve": {

```

```

"phonetic": "/ɪn'vɒlv/",
"meaning": "涉及",
"level": "junior"
},
"concern": {
"phonetic": "/kən'sɜ:n/",
"meaning": "涉及",
"level": "junior"
},
"issue": {
"phonetic": "/'ɪʃu:/",
"meaning": "问题",
"level": "junior"
},
"topic": {
"phonetic": "/'tɒpɪk/",
"meaning": "话题",
"level": "junior"
},
"affair": {
"phonetic": "/ə'feə(r)/",
"meaning": "事务",
"level": "junior"
},
"event": {
"phonetic": "/ɪ'vent/",
"meaning": "事件",
"level": "junior"
},
"incident": {
"phonetic": "/ɪ'ɪnsɪdənt/",
"meaning": "事件",
"level": "junior"
},
"accident": {
"phonetic": "/'æksɪdənt/",
"meaning": "事故",
"level": "junior"
},
"example": {
"phonetic": "/ɪg'zɑ:mpl/",
"meaning": "例子",
"level": "junior"
},
"instance": {
"phonetic": "/ɪnstəns/",
"meaning": "实例",
"level": "junior"
},
"sample": {

```

```

"phonetic": "'sɑ:mpəl/",
"meaning": "样本",
"level": "junior"
},
"model": {
"phonetic": "'mɒdl/",
"meaning": "模型",
"level": "junior"
},
"nevertheless": {
"phonetic": "/,nevəðə'les/",
"meaning": "尽管如此",
"level": "junior"
},
"indeed": {
"phonetic": "/ɪn'di:d/",
"meaning": "确实",
"level": "junior"
},
"actually": {
"phonetic": "'æktʃuəli/",
"meaning": "实际上",
"level": "junior"
},
"various": {
"phonetic": "'veəriəs/",
"meaning": "各种各样的",
"level": "junior"
},
"diverse": {
"phonetic": "/daɪ'vɜ:s/",
"meaning": "多样的",
"level": "junior"
},
"distinct": {
"phonetic": "/dɪ'stɪŋkt/",
"meaning": "明显的",
"level": "junior"
},
"specific": {
"phonetic": "/spə'sɪfɪk/",
"meaning": "具体的",
"level": "junior"
},
"general": {
"phonetic": "'dʒenrəl/",
"meaning": "一般的",
"level": "junior"
},
"typical": {

```

```

"phonetic": "/ˈtɪpɪkl/",
"meaning": "典型的",
"level": "junior"
},
"unique": {
"phonetic": "/juˈni:k/",
"meaning": "独特的",
"level": "junior"
},
"rare": {
"phonetic": "/reə(r)/",
"meaning": "稀有的",
"level": "junior"
},
"unusual": {
"phonetic": "/ʌnˈju:ʒuəl/",
"meaning": "不寻常的",
"level": "junior"
},
"extremely": {
"phonetic": "/ɪkˈstri:mli/",
"meaning": "极其",
"level": "junior"
},
"highly": {
"phonetic": "/ˈhaɪli/",
"meaning": "高度地",
"level": "junior"
},
"deeply": {
"phonetic": "/ˈdi:pli/",
"meaning": "深深地",
"level": "junior"
},
"widely": {
"phonetic": "/ˈwaɪdli/",
"meaning": "广泛地",
"level": "junior"
},
"closely": {
"phonetic": "/ˈkləʊsli/",
"meaning": "紧密地",
"level": "junior"
},
"roughly": {
"phonetic": "/ˈrʌfli/",
"meaning": "大约",
"level": "junior"
},
"approximately": {

```

```

"phonetic": "/ə'prɒksɪmətli/",
"meaning": "大约",
"level": "junior"
},
"eventually": {
"phonetic": "/ɪ'ventʃuəli/",
"meaning": "最终",
"level": "junior"
},
"gradually": {
"phonetic": "/'grædʒuəli/",
"meaning": "逐渐地",
"level": "junior"
},
"rapidly": {
"phonetic": "/'ræpɪdli/",
"meaning": "迅速地",
"level": "junior"
},
"potential": {
"phonetic": "/pə'tenʃl/",
"meaning": "潜力",
"level": "junior"
},
"capacity": {
"phonetic": "/kə'pæsəti/",
"meaning": "能力",
"level": "junior"
},
"approach": {
"phonetic": "/ə'prəʊtʃ/",
"meaning": "方法",
"level": "junior"
},
"strategy": {
"phonetic": "/'strætədʒi/",
"meaning": "策略",
"level": "junior"
},
"tactic": {
"phonetic": "/'tæktɪk/",
"meaning": "战术",
"level": "junior"
},
"hypothesis": {
"phonetic": "/haɪ'pɒθ əsɪs/",
"meaning": "假设",
"level": "junior"
},
"analysis": {

```

```

"phonetic": "/əˈnæləsis/",
"meaning": "分析",
"level": "junior"
},
"synthesis": {
"phonetic": "/ˈsɪnθ əsɪs/",
"meaning": "综合",
"level": "junior"
}
}

// File: app/src/test/java/com/liudong/bookread/ExampleUnitTest.kt
package com.liudong.bookread

import org.junit.Test
import org.junit.Assert.*

/**
 * Example local unit test, which will execute on the development machine (host).
 *
 * See [testing documentation](http://d.android.com/tools/testing).
 */
class ExampleUnitTest {
    @Test
    fun addition_isCorrect() {
        assertEquals(4, 2 + 2)
    }
}

// File: app/src/androidTest/java/com/liudong/bookread/ExampleInstrumentedTest.kt
package com.liudong.bookread

import androidx.test.platform.app.InstrumentationRegistry
import androidx.test.ext.junit.runners.AndroidJUnit4
import org.junit.Test
import org.junit.runner.RunWith
import org.junit.Assert.*

/**
 * Instrumented test, which will execute on an Android device.
 *
 * See [testing documentation](http://d.android.com/tools/testing).
 */
@RunWith(AndroidJUnit4::class)
class ExampleInstrumentedTest {
    @Test
    fun useAppContext() {
        // Context of the app under test.
        val appContext = InstrumentationRegistry.getInstrumentation().targetContext
        assertEquals("com.liudong.bookread", appContext.packageName)
    }
}

```